

Filament Texture资源释放机制

简述

在[native内存占用问题分析文档](#)中，我们提到，回调经过一系列中转转换，最终保存到内部一个vector中，在实际调用过程中，再由内向外一层层调用，直至调用到最外层自定义的回调。

之前的分析并没有将filament内部如何保存纹理buffer并如何释放的过程做详细解释。

这里就以一个简单示例来模拟并分析整个过程的原理。

首先，将测试代码全贴出来，然后再详细分析：

```
#include <vector>
#include <thread>
#include <chrono>
#include <memory>
#include <iostream>

class Buffer
{
public:
    using BufferCallback = void (*)(void *);

    Buffer(const Buffer &rhs) = delete;
    Buffer& operator = (const Buffer &rhs) = delete;

    Buffer(char *data, BufferCallback callback) : dataBuffer(data), bufferCallback(callback) {
        std::cout << "Buffer::Buffer(char *, BufferCallback)" << std::endl;
    }

    ~Buffer() {
        std::cout << "Buffer::~~Buffer()" << std::endl;
        if (bufferCallback) {
            bufferCallback(dataBuffer);
        }
    }

    Buffer(Buffer&& rhs) noexcept
        : dataBuffer(rhs.dataBuffer), bufferCallback(rhs.bufferCallback)
    {
        std::cout << "Buffer::Buffer(Buffer&&)" << std::endl;
        rhs.bufferCallback = nullptr;
    }

private:
    char *dataBuffer;
    BufferCallback bufferCallback;
};

class CallbackHandler
{
public:
    using Callback = void (*)(void *user);
};

class Driver;
class Texture
{
public:
    Texture(Driver *drv) : driver(drv) {
        std::cout << "Texture::Texture()" << std::endl;
    }

    ~Texture() {
        std::cout << "Texture::~~Texture()" << std::endl;
    }

    void setImage(Buffer&& buffer);
};
```

```

private:
    Driver *driver;
};

class Driver
{
public:
    Driver() {
        std::cout << "Driver::Driver()" << std::endl;
    }

    ~Driver() {
        std::cout << "Driver::~~Driver()" << std::endl;
    }

    void scheduleDestroy(Buffer&& buffer) {
        scheduleDestroy_Slow(std::move(buffer));
    }

private:
    void scheduleDestroy_Slow(Buffer&& buffer) {
        std::cout << "Driver::scheduleDestroy_Slow()" << std::endl;

        std::cout << "schedule lambda1" << std::endl;
        scheduleCallback([buffer = std::move(buffer)]() {
            std::cout << "Driver::scheduleDestroy_Slow() lambda1" << std::endl;
        });
    }

    void scheduleCallback(void *user, CallbackHandler::Callback callback) {
        std::cout << "Driver::scheduleCallback(int, int, CallbackHandler::Callback)" << std::endl;
        mCallbacks.emplace_back(user, callback);
    }

    struct CallbackData {
        CallbackData(CallbackData const &) = delete;
        CallbackData(CallbackData &&) = delete;
        CallbackData& operator=(CallbackData const &) = delete;
        CallbackData& operator=(CallbackData const &&) = delete;

        CallbackData() {
            std::cout << "Driver::CallbackData::CallbackData()" << std::endl;
        }

        ~CallbackData() {
            std::cout << "Driver::CallbackData::~~CallbackData()" << std::endl;
        }

        void *storage[8] = {};
        static void release(CallbackData *data) { delete data; };
    };

    template <typename T>
    void scheduleCallback(T&& functor) {
        std::cout << "template <typename T> Driver::scheduleCallback(int, int, T&&)" << std::endl;
        CallbackData *data = new CallbackData;

        std::cout << "sizeof(functor): " << sizeof(functor) << std::endl;
        new(data->storage) T(std::forward<T>(functor)); //CallbackDatastoragelambda1

        std::cout << "schedule lambda2" << std::endl;
        scheduleCallback(data, (CallbackHandler::Callback)[](void *data) {
            std::cout << "template <typename T> Driver::scheduleCallback(int, int, T&&) lambda2" <<
std::endl;

            CallbackData *mydata = static_cast<CallbackData *>(data);
            void *user = mydata->storage;
            T& functor = *static_cast<T*>(user);
            functor(); //lambda1
            functor.~T(); //lambda1
            CallbackData::release(mydata); //newCallbackData

```

```

        });
    }

public:
    void purge() noexcept {
        std::cout << "Driver::purge()" << std::endl;

        decltype(mCallbacks) callbacks;
        std::swap(callbacks, mCallbacks);

        std::cout << "Sizeof callbacks: " << callbacks.size() << std::endl;

        for (auto &item : callbacks) {
            item.second(item.first);
        }
    }

private:
    std::vector<std::pair<void *, CallbackHandler::Callback>> mCallbacks;
};

void Texture::setImage(Buffer&& buffer) {
    std::cout << "Texture::setImage(Buffer&&)" << std::endl;

    driver->scheduleDestroy(std::move(buffer));
}

void gBufferCallback(void *data)
{
    std::cout << " ----- gBufferCallback -----" << std::endl;

    if (data) {
        std::cout << " ----- Release Buffer -----" << std::endl;

        char *buffer = (char *)data;
        delete []buffer;

        std::cout << "Buffer Released" << std::endl;
    }
}

int main()
{
    constexpr static int textureBufferSize = 1024;

    std::cout << "----- Init Engine -----" << std::endl;

    std::cout << "Driver" << std::endl;
    Driver *driver = new Driver();

    std::cout << "Texture" << std::endl;
    Texture *texture = new Texture(driver);

    std::cout << "----- Prepare Buffer -----" << std::endl;
    char *buffer = new char[textureBufferSize];
    std::cout << "Buffer Created" << std::endl;

    std::cout << "----- Create Texture -----" << std::endl;

    std::cout << "setImage" << std::endl;
    texture->setImage({buffer, gBufferCallback});

    std::cout << std::endl;

    std::cout << "----- Render Frame -----" << std::endl;

    #if 0 //0, 1
    std::cout << " ----- Render -----" << std::endl;
    driver->purge();
    std::cout << std::endl;
    #endif
}

```

```

#else
    std::cout << " ----- No Render Frame -----" << std::endl;
#endif

    std::cout << "----- Render End -----" << std::endl;

    std::cout << std::endl;

    //std::this_thread::sleep_for(std::chrono::seconds(4));

    std::cout << "----- Exit Engine -----" << std::endl;

    delete driver;
    delete texture;

    return 0;
}

```

代码中，我们通过第194行的宏判断(#if)来模拟是否渲染一帧数据（1为渲染，0为不渲染）。

下图就是通过渲染（左）和不渲染（右）两种情况代码运行的结果对比：

```

liwei66@xiaomi:~/work/test$ ./a.out
----- Init Engine -----
创建Driver
Driver::Driver()
创建Texture
Texture::Texture()
----- Prepare Buffer -----
Buffer Created
----- Create Texture -----
setImage
Buffer::Buffer(char *, BufferCallback)
Texture::setImage(Buffer&&)
Driver::scheduleDestroy_Slow()
schedule lambda1
Buffer::Buffer(Buffer&&)
template <typename T> Driver::scheduleCallback(int, int, T&&)
Driver::CallbackData::CallbackData()
Buffer::Buffer(Buffer&&)
schedule lambda2
Driver::scheduleCallback(int, int, CallbackHandler::Callback)
Buffer::~Buffer()
Buffer::~Buffer()

----- Render Frame -----
----- Render -----
Driver::purge()
Sizeof callbacks: 1
template <typename T> Driver::scheduleCallback(int, int, T&&) lambda2
Driver::scheduleDestroy_Slow() lambda1
Buffer::~Buffer()
----- gBufferCallback -----
----- Release Buffer -----
Buffer Released
Driver::CallbackData::~CallbackData()
----- Render End -----
----- Exit Engine -----
Driver::~Driver()
Texture::~Texture()

liwei66@xiaomi:~/work/test$ ./a.out
----- Init Engine -----
创建Driver
Driver::Driver()
创建Texture
Texture::Texture()
----- Prepare Buffer -----
Buffer Created
----- Create Texture -----
setImage
Buffer::Buffer(char *, BufferCallback)
Texture::setImage(Buffer&&)
Driver::scheduleDestroy_Slow()
schedule lambda1
Buffer::Buffer(Buffer&&)
template <typename T> Driver::scheduleCallback(int, int, T&&)
Driver::CallbackData::CallbackData()
Buffer::Buffer(Buffer&&)
schedule lambda2
Driver::scheduleCallback(int, int, CallbackHandler::Callback)
Buffer::~Buffer()
Buffer::~Buffer()

----- Render Frame -----
----- No Render Frame -----
----- Render End -----
----- Exit Engine -----
Driver::~Driver()
Texture::~Texture()
liwei66@xiaomi:~/work/test$
liwei66@xiaomi:~/work/test$
liwei66@xiaomi:~/work/test$
liwei66@xiaomi:~/work/test$
liwei66@xiaomi:~/work/test$
liwei66@xiaomi:~/work/test$
liwei66@xiaomi:~/work/test$
liwei66@xiaomi:~/work/test$
liwei66@xiaomi:~/work/test$
liwei66@xiaomi:~/work/test$

```

图中我们可以看到，demo经历了如下几个过程：

- Init Engine
- Prepare Buffer
- Create Texture
- Render Frame
- Render End
- Exit Engine

对比左右两边，唯一不同的就是左边Render Frame和Render End之间（也就是渲染一帧过程）实际执行了渲染过程Render并调用了purge，之后就依次经历了两个lambda中转（Driver::scheduleDestroy_Slow和Driver::scheduleCallback中的临时lambda，

我们分别叫做lambda1和lambda2），并调用了我们自定义的实际回收内存的函数（gBufferCallback）来实际清理分配内存，而右侧并没有实际的渲染和purge调用，也就导致了lambda1、lambda2都没用调用，进而自定义的实际回收内存的函数（gBufferCallback）

也就没有调用，实际分配的内存得不到释放。

详细分析

下面，通过各个阶段，来逐步分析每个部分的重点，并在适当的位置来对核心实现原理进行详解。

Init Engine

这部分主要就是创建了Driver对象和Texture对象，为后续处理过程做准备。

Driver类定义：

```
class Driver
{
public:
    ...
    void scheduleDestroy(Buffer&& buffer) {
        scheduleDestroy_Slow(std::move(buffer));
    }

private:
    void scheduleDestroy_Slow(Buffer&& buffer) {
        ...
        scheduleCallback([buffer = std::move(buffer)]() {
            ...
        });
    }

    void scheduleCallback(void *user, CallbackHandler::Callback callback) {
        ...
        mCallbacks.emplace_back(user, callback);
    }

    struct CallbackData {
        ...
        void *storage[8] = {};
        static void release(CallbackData *data) { delete data; };
    };

    template <typename T>
    void scheduleCallback(T&& functor) {
        ...
        CallbackData *data = new CallbackData;
        new(data->storage) T(std::forward<T>(functor));
        scheduleCallback(data, (CallbackHandler::Callback)[](void *data) {
            ...
            functor();
            functor.~T();
            CallbackData::release(mydata); //newCallbackData
        });
    }

public:
    void purge() noexcept {
        ...
        std::swap(callbacks, mCallbacks);
        ...
        for (auto &item : callbacks) {
            item.second(item.first);
        }
    }

private:
    std::vector<std::pair<void *, CallbackHandler::Callback>> mCallbacks;
};
```

Driver类中，主要定义了如下内容：

- 一个对外接口scheduleDestroy，用来调度外部资源buffer的释放。
- 一个私有的vector（mCallbacks）用来保存渲染过程需要的资源回调（CallbackHandler::Callback）及其参数（void *），并在purge中进行遍历和调用。
- 内部调度资源buffer释放的scheduleDestroy_Slow，将传入的buffer对象移动到lambda1的跟踪参数中，从而将其声明周期延长到和lambda1之后，并将lambda1传递给lambda2（作为Driver::scheduleCallback(T&& functor)参数传入）。
- 实际用来保存资源回调的scheduleCallback，将资源buffer回调存入vector。
- 一个用来保存lambda2的数据结构CallbackData，内部实现了用来释放自己的release函数。

- 一个用来保存lambda2的模板函数scheduleCallback(T&& functor)，用来调用scheduleCallback(void *user, CallbackHandler::Callback callback)将回调和参数存入vector。
- 一个用来释放资源buffer的公共purge函数，遍历vector，依次释放各个资源buffer。

我们在main函数中，简单将Texture和Driver定义成依赖关系，并简单new两个类的对象，这里只是示意，实际代码中远比这个过程复杂，这里只是为演示后面过程做准备。

Prepare Buffer

这里只是简单new了一个数组，并将其传入Buffer包装类，并在Buffer对象的后续传递过程(实际的buffer是在具体驱动如Vulkan的VulkanDriver::update3DImage中将其拷贝到对应的Image对象中的)中消费的。

Create Texture

这一步，其实对应到真实代码中，实际上就是对Ktx1Reader::createTexture这一调用的模仿，这里其实就是通过刚刚new出来的Texture来将上步创建的buffer传入setImage函数，实际代码就是将Ktx1Texture的数据buffer设置进驱动中。这里

简单地用上一步创建的buffer和自定义的buffer释放回调(gBufferCallback)创建了一个Buffer类的临时对象（构造过程会将传入的buffer和gBufferCallback分别保存到成员dataBuffer和bufferCallback中，以供析构时候调用），并将其作为参数传入Texture::setImage，由于setImage的参数是Buffer&&类型，也就是右值引用类型，刚刚的临时对象在参数传递过

程中就不会因为临时对象拷贝而浪费效率，也同时在后续流程通过move将对象转移，这样就不会由于临时对象周期结束而丢失内部数据。从而达到和将临时变量声明周期延长相当的效果。其实这和智能指针unique_ptr所有权转移的过程很像。

最终，在setImage函数调用结束，这个临时Buffer就会被释放，这时Buffer对象内部数据实际上已经由于后面的move操作被置空了。

```
void Texture::setImage(Buffer&& buffer) {
    std::cout << "Texture::setImage(Buffer&&)" << std::endl;

    driver->scheduleDestroy(std::move(buffer));
}
```

其内部实际上就是调用了Driver::scheduleDestroy(Buffer&&)来完成后续Buffer对象的传递的。

```
Texture::setImage(Buffer&& buffer)
-->scheduleDestroy(Buffer&& buffer)
---->scheduleDestroy_Slow(std::move(buffer));
----->scheduleCallback([buffer = std::move(buffer)]() { });
```

上面是后续调用流程和相关参数，可以看到，最后一个调用，将buffer移动到了一个临时buffer对象中，这个临时对象又和这个空lambda表达式（lambda1）生命周期想关联，当lambda1声明周期结束，临时对象生命也就结束。

```
template <typename T>
void scheduleCallback(T&& functor) {
    ...
    CallbackData *data = new CallbackData;
    new(data->storage) T(std::forward<T>(functor));
    scheduleCallback(data, (CallbackHandler::Callback)[](void *data) {
        ...
        functor();
        functor.~T();
        CallbackData::release(mydata); //newCallbackData
    });
}
```

上面最后一步，调用了模板函数scheduleCallback，并传入了一个lambda表达式lambda1，在这里，我们通过placement new，将lambda1保存到了CallbackData对象data的storage中。这里，我们实际上是将lambda1使用完美转发的方式传入placement new中，这

就会导致lambda1中的Buffer对象又进行了一次move，这时，Buffer就被置空，当函数调用完成这个lambda1对象声明周期也就结束了，空Buffer也就会销毁。

然后将data和临时lambda（lambda2，强转为CallbackHandler::Callback callback类型）传入Driver::scheduleCallback(void *, CallbackHandler::Callback)中，并最终将他们作为一个std::pair存入vector中（mCallbacks）。

```
void scheduleCallback(void *user, CallbackHandler::Callback callback) {
    std::cout << "Driver::scheduleCallback(int, int, CallbackHandler::Callback)" << std::endl;
    mCallbacks.emplace_back(user, callback);
}
```

同样，当`scheduleCallback`调用结束，传入的临时对象`lambda2`声明周期也就结束了，而`vector`中保存的是另外的一份拷贝。

此时，调用完成后，函数会逐层返回，并最终从`setImage`返回。这个过程中两次`Buffer`移动创建了两个新`buffer`，加上最初的临时对象，一共三个`Buffer`对象，在函数返回过程中会释放两个，最终的那个最新的`Buffer`对象随着`lambda1`被保存进`vector`中。当`lambda1`回调结束后，也就

被释放了。

实际上`Create Texture`过程，就是`Buffer`交换传递过程和几个临时`lambda`表达式传递保存的过程。

Render Frame

在这个过程中，我们通过一个宏判断来模拟是否走了渲染过程，实际就是是否走`Driver::purge`的区别。

```
std::cout << "----- Render Frame -----" << std::endl;

#if 1 //0, 1
    std::cout << " ----- Render -----" << std::endl;
    driver->purge();
    std::cout << std::endl;

#else
    std::cout << " ----- No Render Frame -----" << std::endl;
#endif
    std::cout << "----- Render End -----" << std::endl;
```

`purge`定义如下：

```
void purge() noexcept {
    std::cout << "Driver::purge()" << std::endl;

    decltype(mCallbacks) callbacks;
    std::swap(callbacks, mCallbacks);

    std::cout << "Sizeof callbacks: " << callbacks.size() << std::endl;

    for (auto &item : callbacks) {
        item.second(item.first);
    }
}
```

当调用`purge`时，实际上就是遍历`mCallbacks`的过程，为什么不直接遍历，而是将`mCallbacks`交换给`callbacks`呢？

这里就是新标准C++中对RAII使用的典型场景了。

如果我们直接遍历`mCallbacks`，那么遍历结束，`mCallbacks`还是依然存在的，我们就不得不对它进行手动释放，而实际代码在这里是对`mCallbacks`加锁的，这势必会将加锁时间延长，并且手动去主动释放对象也并不是新标准C++提倡的。

这时，我们通过在栈里声明了一个临时对象`callbacks`，并将`mCallbacks`交换给它，这样，原来的`mCallbacks`就为空了，不需要手动释放，而在`callbacks`在遍历后，函数返回，这时`callbacks`声明周期结束，就会被自动删除回收。

从上节最后我们知道，`mCallbacks`里保存的是`lambda2`的一个拷贝（`item.second`），还有`CallbackData`的指针`data(item.first)`，所以遍历的过程实际上就是依次执行：

```
[(void *data) {
    std::cout << "template <typename T> Driver::scheduleCallback(int, int, T&&) lambda2" << std::endl;

    CallbackData *mydata = static_cast<CallbackData *>(data);
    void *user = mydata->storage;
    T& functor = *static_cast<T*>(user);
    functor(); //lambda1
    functor.~T(); //lambda1
    CallbackData::release(mydata); //newCallbackData
}];
```

我们只调用了一次`setImage`，也就是`vector`中只有一项。

这里首先将`data`转换成`CallbackData`指针，这和我们存入的是一个类型，转换过程不会有问題，这里只是示意，并没有进行有效性等检测。

接着，我们在其中将storage取出，由于其保存的是lambda1的仿函数，我们通过调用这个仿函数就可以调用到lambda1，而lambda1实际只是占位，内容为空：

```
[buffer = std::move(buffer)]() {
    std::cout << "Driver::scheduleDestroy_Slow() lambda1" << std::endl;
};
```

当lambda1调用完成，调用了仿函数的析构函数将其释放：

```
functor.~T(); //lambda1
```

这时，lambda1销毁，接着，它内部的临时Buffer对象生命周期也就结束，会调用到其析构函数释放Buffer对象内存：

```
~Buffer() {
    std::cout << "Buffer::~~Buffer()" << std::endl;
    if (bufferCallback) {
        bufferCallback(dataBuffer);
    }
}
```

通过上面Create Texture最初的分析，我们知道，Buffer内部保存了数据buffer（dataBuffer）和传入的释放buffer的函数指针（bufferCallback），这里析构实现就是直接将数据buffer作为参数调用buffer释放函数。

这样就在释放Buffer过程调用了我们自定义的释放函数gBufferCallback(void *)。

这就是正常情况下释放自定义缓冲区的过程。在没有渲染数据的情况下，purge函数不会调用，也就不会遍历mCallbacks内部成员来调用其中的lambda，也就没有释放Buffer的析构过程，进而也就不会释放我们自定义的buffer。

Exit Engine

这里只是简单反向依次释放driver和texture，过程和创建过程反向即可。这里没有实际影响流程的内容。简单带过。

总结

以上就是完整的创建和释放过程详细描述。代码只是简单示意，没有有效性判断，没有线程安全考虑，只为演示基本原理。

编译过程记得加上-std=c++14或更高版本。